WorkTrack 项目开发指南

以下是一份完整的、面向 AI 辅助开发的 **WorkTrack 项目开发指南**。你可以将本文档直接提供给 ChatGPT、Claude、Cursor 或其他 AI 工具，分阶段请求生成代码，确保不偏离整体架构和设计。

WorkTrack 项目完整开发指南

> 版本：1.0
>
> 适用范围：个人部署于 NAS 的 Docker 化工作管理平台，支持日报、客户项目、会议纪要，集成 AI 能力与 MCP 服务

---

1. 项目概述

WorkTrack 是一款自托管的个人工作管理工具，核心功能包括：

- 日常工作日报记录（Markdown 编辑、AI 整理）
- 客户与项目管理（跟进记录、状态看板）
- 会议纪要管理（结构化提取、关联客户/项目）
- 向量化数据管理（语义搜索、RAG）
- 多模型 AI 集成（OpenAI 兼容接口，手动/自动整理，定时任务）
- MCP 服务能力（对外暴露标准工具，供 OpenClaw 等智能体调用）

部署方式：Docker 单容器，数据持久化于 NAS 目录。

---

2. 技术栈与全局配置

2.1 技术选型

层级	技术选型	说明
后端框架	Python 3.11+ + FastAPI	异步高性能、AI 库生态丰富
数据库	SQLite + SQLModel	单文件、零运维，WAL 模式开启
向量数据库	Chroma（内嵌模式）	轻量持久化，底层基于 SQLite
任务调度	APScheduler + SQLAlchemyJobStore	持久化定时任务，Cron 支持
文件存储	本地文件系统（NAS 挂载卷）	路径存入数据库
前端	Next.js 14 (App Router) 或 React + Vite	静态部署，轻量快速
样式	Tailwind CSS + shadcn/ui	统一设计系统，组件可复制
图标	Lucide React	线性风格，暗色主题友好
AI 接入	openai 库（兼容 OpenAI 协议）	支持 DeepSeek、通义千问等
MCP 服务	fastmcp 或自研轻量 MCP Server	标准 MCP 工具暴露
容器化	Docker + Docker Compose	一键部署

2.2 项目目录结构

```
worktrack/
├── backend/                # Python 后端
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py         # FastAPI 应用入口
│   │   ├── config.py       # 配置管理（pydantic-settings）
│   │   ├── database.py     # 数据库引擎与初始化
│   │   ├── models/         # SQLModel 数据模型
│   │   ├── routers/        # API 路由
│   │   ├── services/       # 业务逻辑（AI、向量、调度等）
│   │   └── mcp_server.py   # MCP 工具定义
│   ├── data/               # 持久化数据（.db, chroma_data, 上传文件）
│   ├── requirements.txt
│   ├── Dockerfile
│   └── .env.example
├── frontend/               # 前端项目（Next.js 或 Vite）
│   ├── src/
│   │   ├── components/     # shadcn/ui 组件
│   │   ├── pages/          # 页面
│   │   └── styles/         # 全局样式
│   ├── tailwind.config.js
│   └── package.json
├── docker-compose.yml
└── README.md
```

2.3 环境变量模板（`.env.example`）

代码块

```
# 对话模型
LLM_BASE_URL=https://api.deepseek.com/v1
LLM_API_KEY=sk-your-key
LLM_MODEL_NAME=deepseek-chat

# Embedding 模型（若不单独配置则自动复用对话模型配置）
EMBEDDING_BASE_URL=
EMBEDDING_API_KEY=
EMBEDDING_MODEL_NAME=text-embedding-3-small

# Chroma 持久化目录
CHROMA_PERSIST_DIR=./data/chroma

# 数据库路径
```

```
DATABASE_URL=sqlite:///./data/worktrack.db
```

2.4 Docker Compose 模板

代码块

```yaml
version: '3.8'
services:
  worktrack:
    build: ./backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend/data:/app/data       # 数据库、向量、文件
      - ./backend/.env:/app/.env       # 环境变量
    environment:
      - LLM_BASE_URL=${LLM_BASE_URL}
      - LLM_API_KEY=${LLM_API_KEY}
      - LLM_MODEL_NAME=${LLM_MODEL_NAME}
      - EMBEDDING_BASE_URL=${EMBEDDING_BASE_URL}
      - EMBEDDING_API_KEY=${EMBEDDING_API_KEY}
      - EMBEDDING_MODEL_NAME=${EMBEDDING_MODEL_NAME}
    restart: unless-stopped
```

---

3. 数据库模型设计（SQLModel）

3.1 表定义

所有模型位于 `backend/app/models/`，继承 `SQLModel`。

用户表

```
class User(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    avatar: Optional[str] = None
```

日报表

代码块

```
class DailyReport(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    user_id: int = Field(foreign_key="user.id")
    report_date: date
    content_md: str
    ai_summary: Optional[str] = None
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)
```

客户表

代码块

```
class Customer(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    industry: Optional[str] = None
    contact: Optional[str] = None
    status: str = "潜在"  # 潜在/接洽中/已签约/维护中
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

项目表

代码块

```
class Project(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    customer_id: int = Field(foreign_key="customer.id")
    name: str
    status: str = "进行中"  # 进行中/已完成/暂停
    deadline: Optional[date] = None
```

```
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

会议纪要表

代码块

```
class MeetingNote(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    customer_id: Optional[int] = Field(foreign_key="customer.id")
    project_id: Optional[int] = Field(foreign_key="project.id")
    title: str
    content_md: str
    attendees: Optional[str] = None  # JSON 字符串
    meeting_date: datetime
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

定时任务表

代码块

```
class ScheduledTask(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    trigger_type: str  # cron/interval/date
    trigger_config: str  # JSON: {cron: "...", hour: ...}
    action_type: str     # ai_summarize_daily / ai_analyze_project 等
    action_params: Optional[str] = None  # JSON 附加参数
    enabled: bool = True
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

3.2 数据库初始化

`backend/app/database.py`：

代码块

```
from sqlmodel import create_engine, SQLModel, Session
engine = create_engine("sqlite:///./data/worktrack.db", echo=False)

```

```python
def init_db():
    from app.models import user, daily_report, customer, project, meeting_note, scheduled_task
    SQLModel.metadata.create_all(engine)
    with engine.connect() as conn:
        conn.execute("PRAGMA journal_mode=WAL")
        conn.execute("PRAGMA foreign_keys=ON")

def get_session():
    with Session(engine) as session:
        yield session
```

在 `main.py` 启动时调用 `init_db()`。

---

4. 后端 API 路由设计（FastAPI）

4.1 统一前缀与依赖

所有业务路由挂载在 `/api/v1` 下，通过 `app.routers` 组织。每个路由文件使用 `APIRouter`，并在 `main.py` 中引入。

依赖注入获取数据库会话：

代码块

```python
from app.database import get_session

def get_db():
    db = next(get_session())
    try:
        yield db
    finally:
        db.close()
```

4.2 日报 CRUD（`routers/daily_reports.py`）

- `GET /api/v1/reports?start_date=&end_date=` - 查询日期范围内的日报
- `POST /api/v1/reports` - 创建日报
- `PUT /api/v1/reports/{id}` - 更新日报
- `DELETE /api/v1/reports/{id}` - 删除日报
- `POST /api/v1/reports/{id}/ai-summarize` - 手动触发 AI 总结，更新 `ai_summary` 字段

所有创建/更新操作后，通过 `BackgroundTasks` 调用向量索引更新（见第5章）。

4.3 客户与项目管理（`routers/customers.py`，`routers/projects.py`）

- 客户 CRUD，列表查询支持按状态过滤
- 项目 CRUD，关联客户，查询支持按客户 ID 过滤
- `GET /api/v1/customers/{id}/overview` - 返回客户详情、关联项目、最近会议列表的聚合视图

4.4 会议纪要管理（`routers/meetings.py`）

- 标准 CRUD
- `POST /api/v1/meetings/{id}/ai-extract` - 调用 AI 从纪要内容中提取结构化信息（决议/待办/结论），返回 JSON 或更新记录的特定字段

4.5 定时任务管理（`routers/scheduled_tasks.py`）

- CRUD 定时任务（仅限管理操作，不会直接执行）
- 后端通过 APScheduler 实际调度，与数据库同步

4.6 AI 对话代理（`routers/ai_agent.py`）

- `POST /api/v1/ai/chat` - 接收用户消息，执行 Function Calling 流程，返回最终回复和工具调用历史

4.7 搜索接口

- `GET /api/v1/search?q=&type=daily_report|customer|meeting` - 混合搜索（全文+语义）
  - 语义搜索部分调用向量服务

---

5. 向量化与语义搜索服务

5.1 向量存储初始化

`backend/app/services/vector_store.py`：

代码块

```
import chromadb
from chromadb.config import Settings
from openai import OpenAI
from app.config import settings

embedding_client = OpenAI(
    base_url=settings.embedding_base_url,
    api_key=settings.embedding_api_key,
)

chroma_client = chromadb.PersistentClient(
    path=settings.chroma_persist_dir,
    settings=Settings(anonymized_telemetry=False)
```

```
)

def get_collection(name: str):
    return chroma_client.get_or_create_collection(name=name)

def embed_text(text: str) -> list[float]:
    resp = embedding_client.embeddings.create(
        model=settings.embedding_model_name,
        input=text
    )
    return resp.data[0].embedding

def index_document(collection_name: str, doc_id: str, text: str, metadata: dict):
    collection = get_collection(collection_name)
    embedding = embed_text(text)
    collection.upsert(
        ids=[doc_id],
        embeddings=[embedding],
        metadatas=[metadata],
        documents=[text]
    )

def search_similar(collection_name: str, query: str, top_k: int = 5, filter_meta: dict = None):
    collection = get_collection(collection_name)
    query_emb = embed_text(query)
    results = collection.query(
        query_embeddings=[query_emb],
        n_results=top_k,
        where=filter_meta
    )
    return results
```

5.2 集成到 CRUD 路由中

在每个创建/更新端点内，添加后台任务：

代码块

```
background_tasks.add_task(
    index_document,
```

```
    collection_name="daily_reports",
    doc_id=str(report.id),
    text=report.content_md,
    metadata={"report_date": str(report.report_date), "user_id": report.user_id}
)
```

同理，为客户、项目、会议纪要建立各自的 collection，如 `customers`，`projects`，`meeting_notes`。

项目描述或客户沟通记录也可单独索引。

---

6. AI 服务与 Function Calling

6.1 AI 客户端初始化

`backend/app/services/ai_service.py`：

代码块

```
from openai import OpenAI
from app.config import settings

llm_client = OpenAI(base_url=settings.llm_base_url, api_key=settings.llm_api_key)
```

6.2 基础 AI 能力

提供同步函数，依赖数据库会话（由调用方传入）：

- `summarize_daily_report(content: str) -> str`
- `extract_meeting_minutes(content: str) -> dict` (使用 JSON mode)
- `generate_project_analysis(project_id: int, db: Session) -> str`

这些函数内部调用 `llm_client.chat.completions.create`，使用系统提示词定制输出。

6.3 AI Agent with Tools

实现 `run_agent_chat(user_message: str, history: list, db: Session) -> str`：

1. 定义工具列表（JSON Schema）：
   - `search_reports(query: str, top_k: int)` -> 调用向量搜索
   - `get_customer_summary(customer_id: int)` -> 从数据库聚合并返回
   - `create_scheduled_task(name: str, cron: str, action_type: str, params: dict)` -> 写入数据库并注册到 APScheduler
   - `summarize_today_reports()` -> 查询今天日报并调用 AI 总结
2. 将用户消息和工具列表发给 LLM
3. 若 LLM 返回 `tool_calls`，在本地执行工具函数，将结果以 `tool` 角色消息追加到对话历史
4. 重复直到 LLM 不再请求工具调用或返回最终文本回复
5. 返回最终回复给前端

6.4 定时任务执行器

定时任务触发时，调用对应的 action，例如 `ai_summarize_daily` 会查询当天日报数据，调用 `summarize_daily_report` 并存储结果。

---

7. MCP 服务实现

使用 fastmcp 或自建轻量 MCP Server，与 FastAPI 运行在同一进程（可通过 `/mcp` 子路径或独立端口暴露）。

示例（`backend/app/mcp_server.py`）：

代码块

```
from fastmcp import FastMCP
from app.services.vector_store import search_similar
from app.database import get_session
from app.models import Customer, DailyReport

mcp = FastMCP("WorkTrack")

@mcp.tool()
def search_reports(query: str, top_k: int = 5) -> list:
    """语义搜索日报内容，返回匹配结果"""
    results = search_similar("daily_reports", query, top_k)
    return [{"id": id_, "metadata": meta} for id_, meta in zip(results['ids'][0], results['metadatas'][0])]

@mcp.tool()
def get_customer_overview(customer_id: int) -> dict:
    """获取客户详细信息及关联项目、最近会议"""
    db = next(get_session())
    customer = db.get(Customer, customer_id)
    # 查询项目、会议等，组装成 dict
    return overview

# 更多工具：create_daily_report, list_tasks 等
```

在 `main.py` 中挂载 MCP（如 `app.mount("/mcp", mcp.http_app())`）。

---

8. 前端 UI 设计系统与页面实现指南

8.1 整体风格

- **暗色模式**：背景色 `#0B0E14`，卡片/侧边栏 `#141820`，悬浮层 `#1E2430`，边框 `#2B3241`
- **主色调**：蓝（`#3B82F6`），辅色：紫（`#8B5CF6`），绿（`#10B981`），橙（`#F59E0B`）
- **字体**：Inter（UI），JetBrains Mono（代码/Markdown）
- **圆角**：卡片 8px，弹窗 12px，按钮全圆角
- **阴影**：柔和，深色背景下用 `ring` 或半透明边框
- **图标**：全部使用 Lucide React，统一线性风格

8.2 基础组件（基于 shadcn/ui）

直接复制并使用 shadcn/ui 的组件：Card, Button, Input, Dialog, DropdownMenu, Skeleton, Command, Tabs, Sheet, Toast。

所有组件默认适配暗色模式（Tailwind 的 dark 类）。

8.3 布局结构

采用三栏布局：

- **左侧固定导航栏**（200px）：Logo、导航项（日报/客户/项目/会议/AI中心/设置），图标+标签，当前页面高亮
- **中间内容区**（自适应，最大宽度 800px）：页面视图动态切换
- **右侧辅助面板**（320px，默认隐藏）：展示详情、AI 侧边栏或快速操作，通过点击条目或 `Ctrl+.]` 打开

移动端：底部 TabBar。

8.4 各页面具体设计

8.4.1 日报页面（`/reports`）

- 顶部：日期选择器 + “今日日报”标题 + “AI 整理”按钮
- 中间：卡片流，按日期分组，每条日报一张卡片，包含日期标签、渲染内容（react-markdown）、标签、操作菜单
- 右下角：蓝色渐变浮动按钮（FAB），点击弹出快速记录抽屉
- 交互：点击卡片展开全屏视图，右侧面板展示关联信息

8.4.2 客户页面（`/customers`）

- 顶部工具栏：搜索、视图切换（列表/看板）、新建客户按钮
- 列表视图：客户行，显示名称、行业、最近跟进、项目数、状态点
- 看板视图：按状态分列（潜在/接洽中/已签约/维护中），卡片可拖拽
- 右侧面板：客户详情，项目列表，会议时间线，AI 分析按钮

8.4.3 会议纪要页面（`/meetings`）

- 列表页：标题、日期、客户、参与人，可展开内联预览
- 详情/编辑页：左右分栏，左侧 Markdown 编辑器（支持 `/` 命令），右侧实时预览，提取结构化块（决议/待办）
- AI 入口：“语音转文字”按钮和“AI 整理”按钮

8.4.4 AI 中心页面（`/ai`）

- 上栏：模型选择器，状态指示灯
- 主区域：聊天界面，用户消息右对齐（蓝背景），AI 左对齐（灰背景），工具调用状态流式展示
- 底部输入框，附加快捷指令按钮（如 `/创建定时任务`）
- 左侧可显示历史会话列表

8.4.5 全局命令面板

`Cmd+K` 弹出，玻璃拟态模糊背景，输入框搜索日报、客户、会议、AI 指令，分组展示，支持键盘导航。

---

9. AI 辅助开发步骤（分阶段实施）

阶段 0：项目初始化

目标：生成项目骨架、Docker 配置、依赖文件。

提示词模板：

> “请根据以下技术栈创建项目目录和配置文件：Python FastAPI 后端、Next.js 前端（可选）、Docker Compose。包含 requirements.txt 和 .env.example。后端目录名为 backend，使用 SQLModel + SQLite + Chroma。请输出完整的文件结构。”

阶段 1：数据库模型与迁移

提示词模板：

> “在 backend/app/models/ 下创建所有 SQLModel 模型，包含 User, DailyReport, Customer, Project, MeetingNote, ScheduledTask。字段如文档所述。同时生成 database.py 实现初始化函数和会话管理。要求开启 SQLite WAL 和外键。”

阶段 2：基础 CRUD API

提示词模板：

> “为 DailyReport 模型创建 FastAPI 路由，支持按日期范围查询、创建、更新、删除。使用 Pydantic Schema 分离请求/响应。包含依赖注入获取数据库会话。同样为客户、项目、会议纪要创建路由。代码放在 backend/app/routers/ 下。”

阶段 3：向量服务集成

提示词模板：

> “实现 vector_store.py，使用 chromadb 持久化客户端和 openai embedding。提供 index_document 和 search_similar 函数。在 CRUD 路由的创建/更新方法中通过 BackgroundTasks 调用 index_document。为不同实体使用不同 collection。”

阶段 4：基础 AI 能力

提示词模板：

> “在 ai_service.py 中实现 summarize_daily_report, extract_meeting_minutes, generate_project_analysis 三个函数，使用 openai 库并读取 LLM 配置。然后在路由中增加手动触发 AI 的端点。”

阶段 5：定时任务调度

提示词模板：

> “集成 APScheduler 到 FastAPI 生命周期，使用 SQLAlchemyJobStore。创建 scheduled_tasks 路由管理任务。任务执行函数暂时留空，后续连接 AI 函数。”

阶段 6：AI Agent 与 Function Calling

提示词模板：

> “实现 run_agent_chat 函数，支持多轮工具调用。定义搜索、获取客户概况、创建定时任务、总结日报等工具。处理 LLM 返回的 tool_calls，执行本地函数并循环。在 ai_agent.py 路由中暴露聊天接口。”

阶段 7：MCP 服务暴露

提示词模板：

> “使用 fastmcp 创建 MCP Server，将向量搜索、客户概况、创建日报等函数暴露为工具。在 main.py 中挂载 MCP 应用。”

阶段 8：前端开发

按页面设计逐步生成。优先实现日报列表、客户列表、AI 聊天界面。每页提示词需包含完整的设计规范描述，可复制第8章中的对应部分。

10. 不变的原则与避免偏离的检查清单

- **所有 AI 调用必须通过统一的配置**，不可硬编码 API Key 或模型名。
- **向量化操作默认异步**（后台任务），不阻塞主线程。
- **数据库迁移必须使用 Alembic**（初次可自动创建表），后续修改模型需生成迁移脚本。
- **前端组件必须复用 shadcn/ui**，不可自行发明不一致的 UI 组件。
- **MCP 工具定义必须与后端服务函数分离**，确保安全性和可维护性。
- **定时任务必须在 APScheduler 中持久化**，重启不丢失。
- **全局使用深色主题**，不可出现浅色硬编码样式。

---

11. 交付物清单

1. 可运行的 Docker 镜像（`docker-compose up` 即启动）
2. 完整的后端 API（含 Swagger 文档）
3. 前端页面（日报、客户、会议、AI 聊天、设置）
4. MCP 服务端点（供 OpenClaw 连接）
5. `.env.example` 文件包含所有必要配置说明
6. `README.md` 包含部署说明和截图

请严格遵循本指南的架构、技术选型和设计系统进行开发，确保每个步骤都符合上述规范，避免项目偏离预定目标。